AMERICAN INTERNATIONAL UNIVERSITY–BANGLADESH (AIUB)

Dept. of Computer Science

Faculty of Science and Technology

CSC4162: PROGRAMMING IN PYTHON

Summer 2025-2025

Section: D

Group No: 04

Project Report On:

Money-wise Tracker: A Personal Expense and Budget Management System

Supervised By: **MD. TANZEEM RAHAT**

Group Members:

Name	ID	Contribution	Department
GAZI SAJID SALMAN	23-55602-3	expense_model.py	CSE
TAMJID NILOY	23-54109-3	expense_manager.py	CSE
TALHA JOBAYER JIHAN	24-59758-3	Interface.py	CSE

GitHub Repository Link: https://github.com/GaziArnob/Python-Summer.git

1. Executive Summary

Moneywise Tracker is a menu-driven Python application developed to help individuals record, organize, and evaluate personal expenses while receiving a simple monthly budget warning. The system uses a structured five-field transaction model, a JSON file for persistent storage, separate model, manager, interface, and launcher modules, and NumPy for descriptive statistical analysis. The sample file supplied contains 10 valid expense records covering eight categories and three months. During verification, the application loaded all records, supported add, view, search, update, delete, save, and load operations, and recovered from invalid menu choices, malformed dates, duplicate IDs, invalid categories, non-numeric amounts, and damaged files without terminating unexpectedly. The sample data produced total spending of 21,950.00, an average expense of 2,195.00, a median of 1,725.00, and a population standard deviation of 1,267.37. Food was the highest spending category at 5,450.00, while June 2026 was the highest month at 9,700.00. Overall, the project satisfies the course emphasis on Python fundamentals, modularity, data structures, file handling, exception handling, OOP, purposeful library use, and meaningful analysis [1].

2. Introduction and Project Objectives

Personal spending is often recorded informally or not recorded at all, which makes it difficult to understand where money is going and whether monthly spending is approaching a chosen limit. MoneyWise Tracker addresses this practical problem with a focused command-line system that accepts transaction data, validates it, stores it permanently, and converts the stored values into useful summaries. The project was deliberately kept small and complete so that its programming logic remains understandable during demonstration and viva.

The intended users are students, early-career earners, and other individuals who need a lightweight offline expense record. The system is not an accounting package; its purpose is to demonstrate reliable personal transaction management and introductory numeric analysis using the concepts required in the Programming in Python course.

Item	Project Content
Application / dataset name	MoneyWise Tracker / expenses.json transaction records
Dataset source	Locally created sample records included with the project; no external dataset is used.
Main project goal	To build a complete Python application that manages expense records, preserves them between sessions, validates normal user input, and produces useful spending and budget summaries.
Research Question 1	How effectively can modular Python logic support reliable expense CRUD operations and persistent JSON storage?
Research Question 2	Which categories and months account for the largest share of spending in the supplied sample records?
Research Question 3	How can descriptive statistics and budget-threshold logic convert raw transactions into practical financial feedback?

2.1 Specific Objectives

- Create a meaningful Expense class and a separate ExpenseManager class.
- Provide at least five meaningful operations through a clear user interface.
- Use list, tuple, set, and dictionary structures for appropriate tasks.
- Store and reload transaction records through readable JSON file handling.
- Prevent duplicate IDs and reject empty, malformed, non-numeric, non-finite, zero, and negative values.
- Use NumPy to calculate total, mean, median, standard deviation, minimum, and maximum expense values.
- Produce category summaries, monthly summaries, unique-category output, and budget-status messages.
- Keep the program running after common user mistakes and file-related problems.

3. Application Data Description

Each row-like JSON object represents one expense transaction. The records are stored as a list of dictionaries in expenses.json and are converted into Expense objects when the application loads. The sample data covers May to July 2026 and is intended to demonstrate all major analysis functions rather than represent a population-level financial study.

3.1 Basic Dataset Information

Property	Value
Number of sample records	10
Number of fields per record	5
Period represented	03 May 2026 to 12 July 2026
Data types present	Text, categorical text, ISO-formatted date text, floating-point numeric
Key variables	expense_id, expense_date, category, amount
Categories represented	8: Bills, Education, Entertainment, Food, Health, Other, Savings, Transport
Potential issues considered	Missing file, empty/corrupted JSON, malformed records, duplicate IDs, invalid category/date, and invalid amount

3.2 Data Dictionary

Field	Type	Validation / Format	Analytical Role
expense_id	String	Unique case-insensitive identifier, e.g., EXP001	Search, update, delete, and duplicate prevention
expense_date	String date	YYYY-MM-DD	Chronological record and monthly grouping
category	Categorical string	One value from the fixed CATEGORIES tuple	Category summary and highest-category calculation
description	String	Non-empty transaction description	Human-readable explanation of the expense
amount	Float	Finite value greater than zero	All totals, comparisons, statistics, and budget checks

3.3 Initial Observations

The sample file contains no missing values, invalid category labels, duplicate IDs, or malformed dates, so all 10 records are loaded successfully. Eight of the eight configured categories appear at least once, giving broad category coverage. Spending is uneven across transactions: the smallest record is 950.00 and the largest is 5,000.00, while monthly totals vary from 4,900.00 to 9,700.00. These differences make the sample suitable for demonstrating descriptive statistics, grouping, and budget messages.

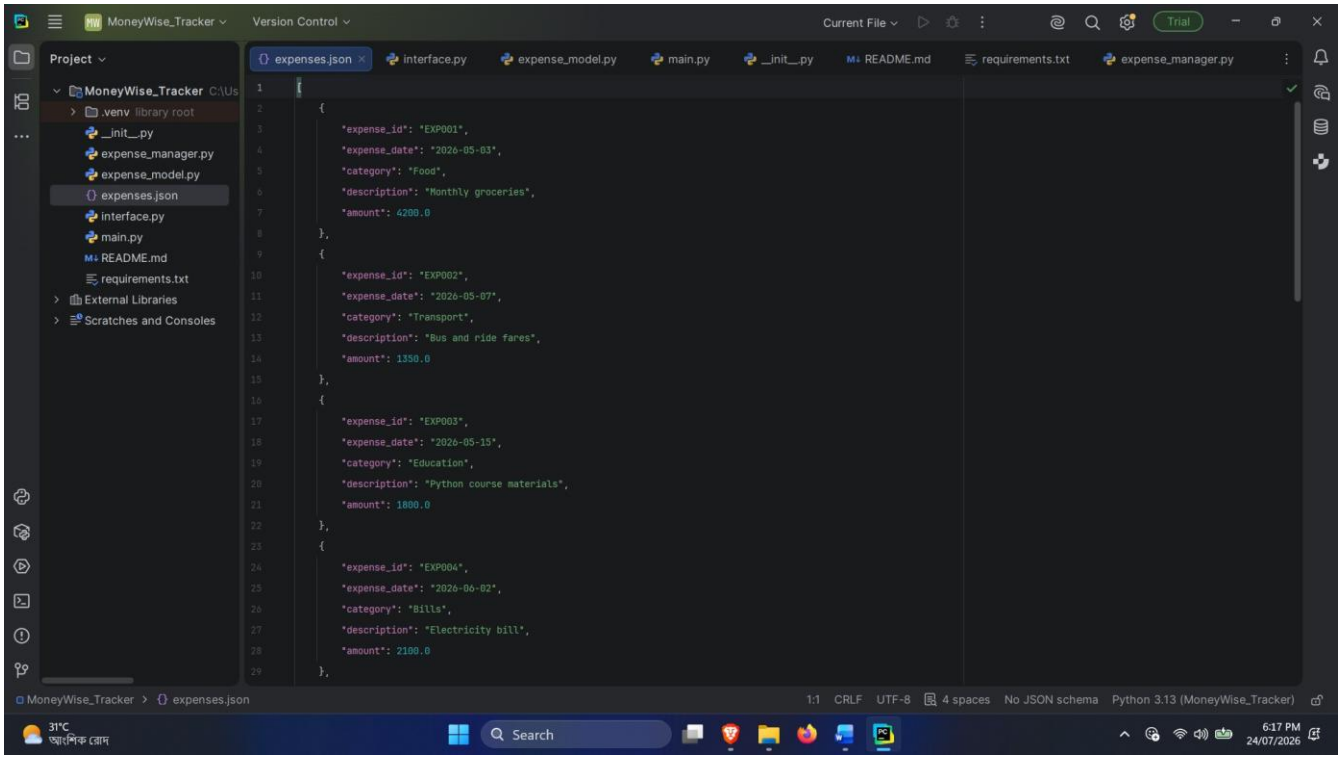

Figure 1. A verified excerpt from the JSON file used for persistent expense storage.

4. Data Validation and Preparation

This project is a transaction-management application rather than a one-time notebook analysis. Consequently, preparation occurs at two points: before a user-created Expense object is accepted and when records are reconstructed from the JSON file. Invalid user values are rejected and requested again; invalid stored records are skipped. The system does not hide missing or incorrect transaction data through imputation because doing so could create false financial records.

Issue Found	Action Taken	Reason / Justification	Code Reference
Blank ID or description	Repeatedly request a non-empty value; the Expense class also checks both fields.	Financial records require an identifier and understandable description.	interface.read_non_empty; Expense.__post_init__
Malformed date	Parse with datetime.strptime using YYYY-MM-DD and reject failures.	Ensures consistent monthly grouping and readable storage.	validate_date; interface.read_date
Non-numeric, zero, negative, NaN, or infinite amount	Convert with float, require isfinite(amount), and require amount > 0.	Prevents impossible values and invalid NumPy/JSON results.	read_positive_amount; Expense.__post_init__
Invalid category	Accept only a numbered selection from the fixed CATEGORIES tuple.	Prevents inconsistent category spelling and grouping.	choose_category; Expense.__post_init__
Duplicate expense ID	Normalize IDs with lower() and check a set before insertion.	Maintains a unique searchable key without case-related duplicates.	used_ids; add_expense
Missing data file	Start with empty collections and create the file on save.	Allows a first-time user to run the application safely.	load_expenses; save_expenses
Empty or corrupted JSON	Catch JSONDecodeError and reset safely to no records.	Prevents startup failure and protects the menu loop.	load_expenses
Partially invalid stored data	Rebuild each Expense inside try-except and count skipped records.	Loads valid transactions while reporting unusable records.	Expense.from_dict; load_expenses

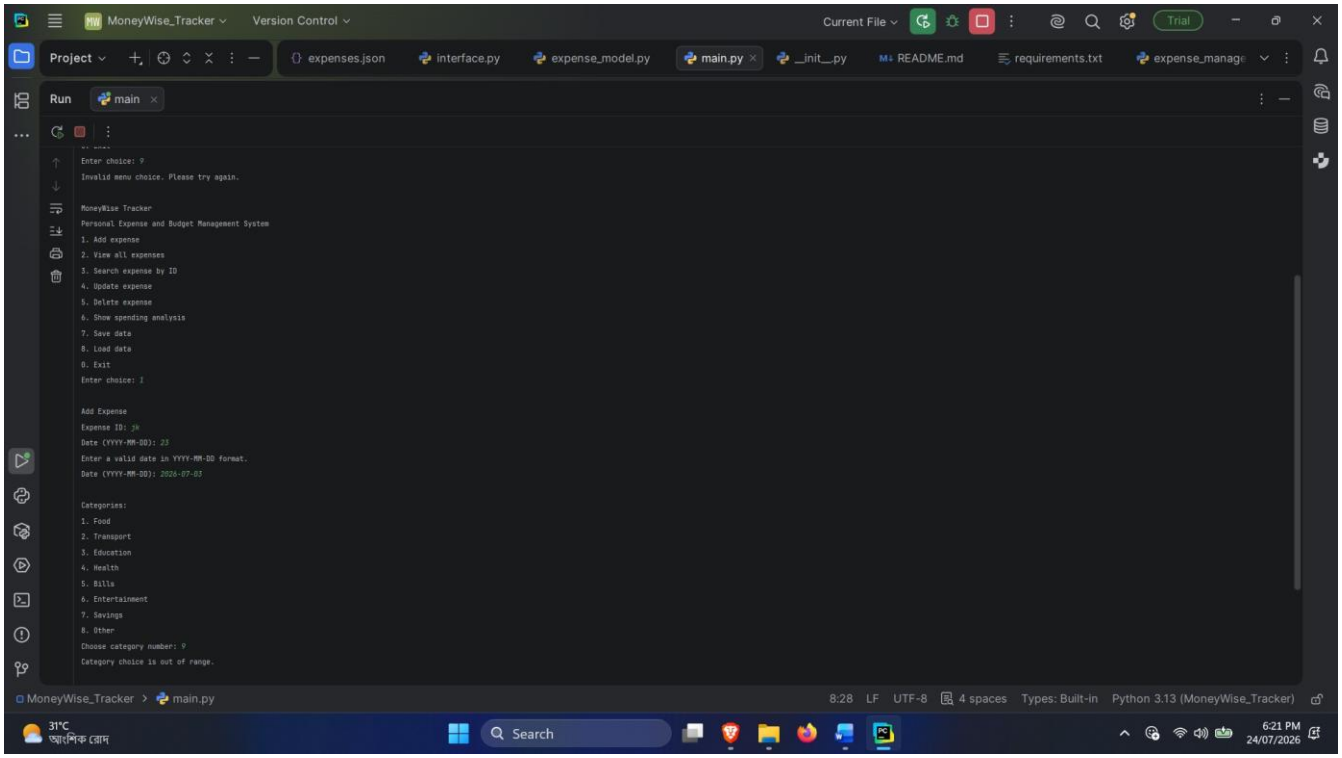

Figure 2. Actual execution evidence showing invalid menu, ID, date, category, description, and amount inputs being handled without a crash.

5. Derived Features and Summary Variables

The project derives analytical values from the original five fields without altering the stored meaning of a transaction. These derived variables are calculated when needed, so the JSON file remains simple and avoids redundant values that could become inconsistent.

Derived Feature	How It Is Created	Why It Is Useful
normalized_id	expense_id.strip().lower()	Supports case-insensitive search and duplicate prevention.
month_key	First seven characters of an ISO date: expense_date [:7]	Groups all transactions belonging to the same calendar month.
category_totals	Dictionary accumulation using totals.get (category, 0) + amount	Measures spending by category and identifies the highest category.
monthly_totals	Dictionary accumulation using month_key	Measures monthly spending and supports the monthly average and budget check.
budget utilization	current_total / monthly_budget × 100	Classify the latest recorded month as good, careful (≥80%), or over budget.
unique_categories	Set comprehension over expense. category	Reports category diversity without duplicate labels.

6. Development and Analysis Methodology

The project follows a requirement-driven modular design. Responsibilities are separated into an entity model, a manager layer, an interface layer, and a small launcher. This structure supports testing, code explanation, and future maintenance better than placing all logic in one large block. The report template supplied by the faculty is generic for Pandas, NumPy, and Matplotlib analysis projects [2]. For this application, Pandas and Matplotlib are intentionally not used: the separate Programming in Python project brief prioritizes Python fundamentals and lists those tools as restricted without prior permission, whereas NumPy is an approved and purposeful analytical library [1].

6.1 Module Structure and Architecture

Project Component	Responsibility
main.py	Creates the manager with a stable path to expenses.json and starts the application.
expense_model.py	Defines fixed tuples, date validation, and the Expense dataclass.
expense_manager.py	Controls records, CRUD operations, file persistence, summaries, NumPy statistics, and budget status.
interface.py	Collects validated input, formats output, displays the menu, and keeps the interaction loop active.
expenses.json	Stores readable persistent sample and user-created records.
requirements.txt	Declares NumPy as the required external dependency.

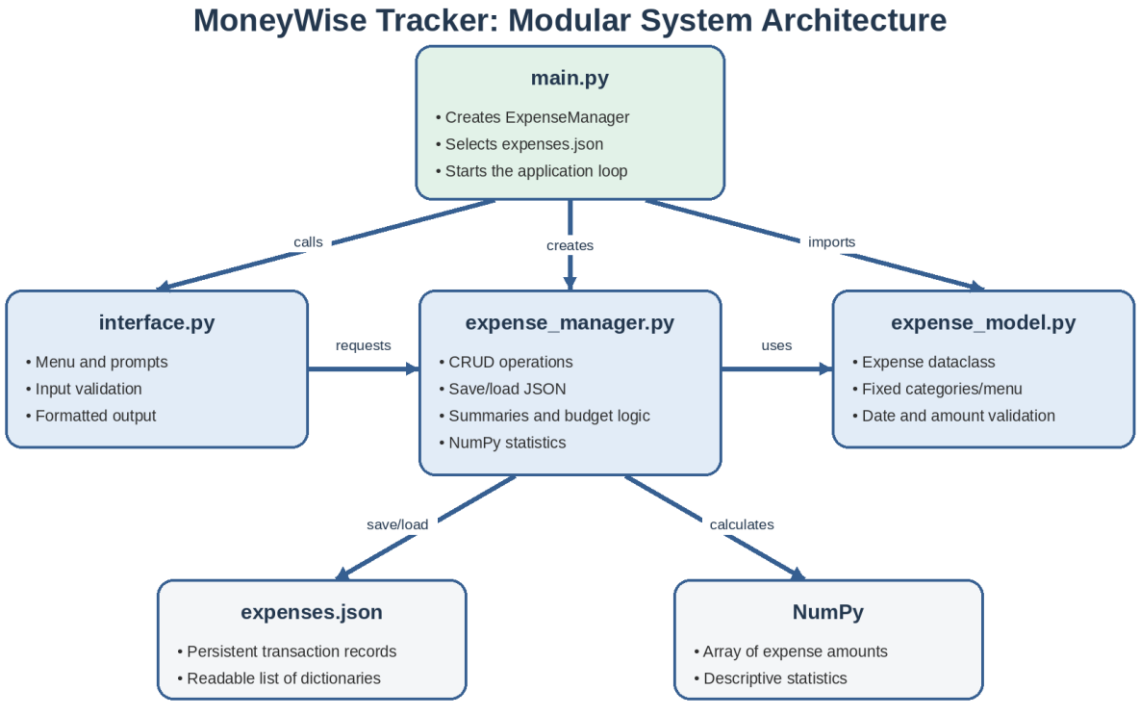

Figure 3. Modular architecture and dependency relationships of MoneyWise Tracker.

6.2 Application Workflow

At startup, main.py resolves the data-file path relative to the project folder, creates ExpenseManager, and calls run_application. The manager attempts to load stored records. The interface then repeats the menu until the user selects exit. Valid operations produce output and, for add, update, or delete, trigger immediate saving. Invalid input produces a clear message and returns control to the relevant prompt or menu.

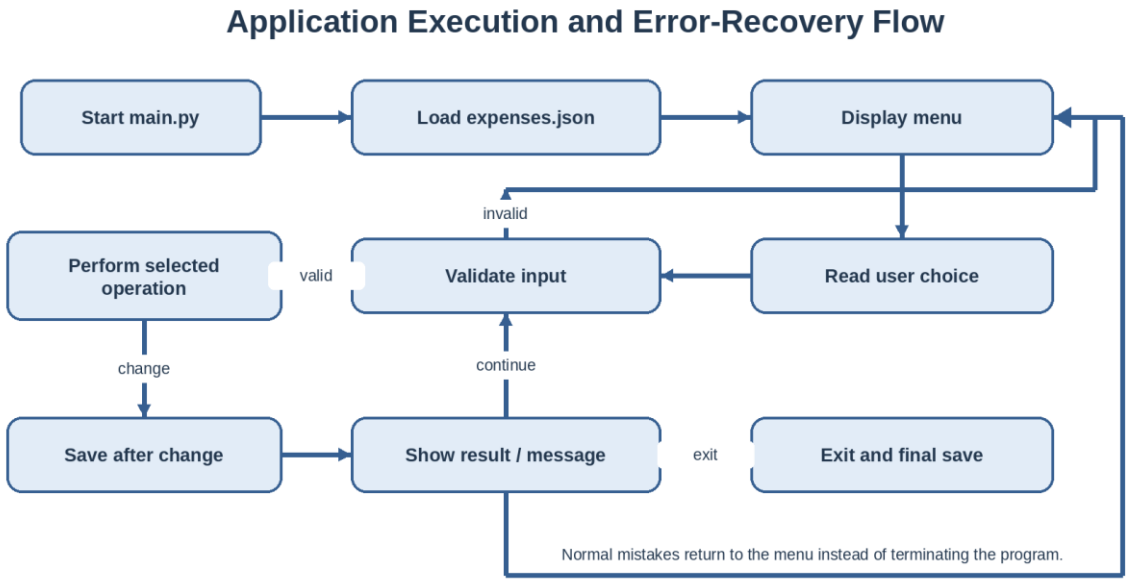

Figure 4. Main execution path, validation loop, save behavior, and error recovery.

6.3 Required Python Concepts

Concept	Evidence in the Project
Variables and data types	IDs, dates, descriptions, categories, amounts, totals, choices, status messages, Path objects, Expense objects.
Operators	Arithmetic totals and remaining budget; comparisons for range, duplicate, and threshold checks; logical conditions for validation.
Branching	if/elif/else menu routing, empty-data checks, budget states, and error messages.
Loops	Persistent menu loop, repeated input validation, record traversal, and summary accumulation.
Functions / methods	Separate input, validation, formatting, CRUD, save/load, summary, statistics, and application-control routines.
List	self.expenses stores multiple Expense objects; loaded JSON begins as a list of records.

Concept	Evidence in the Project
Tuple	CATEGORIES and MENU_OPTIONS store fixed values that should not be modified during execution.
Set	used_ids prevents duplicates; unique_categories removes repeated category labels.
Dictionary	JSON records, category totals, monthly totals, and the statistics result use key-value structures.
File handling	Path.open with JSON load/dump provides persistent and readable records.
Exception handling	ValueError, TypeError, KeyError, JSONDecodeError, and OSError are handled where they can occur.
OOP	Expense models one transaction; ExpenseManager controls a collection and related behavior.
Approved library	NumPy performs real descriptive calculations on the array of expense amounts.

6.4 Object-Oriented Design

Class	Main Attributes	Important Methods	Responsibility
Expense	expense_id, expense_date, category, description, amount	__post_init__, to_dict, from_dict	Represents and validates one real expense transaction.
ExpenseManager	data_file, expenses, used_ids	load_expenses, save_expenses, add_expense, find_expense, update_expense, delete_expense, category_summary, monthly_summary, spending_statistics, budget_status	Controls records, persistence, analysis, and budget feedback.

7. System Operations, Analysis, and Evidence

This section is organized around the three project questions. All numeric values are calculated from the included 10-record expenses.json file. Console figures were captured from an actual tested execution of the completed project.

7.1 Research Question 1: Reliable Operations and Persistence

The application provides eight meaningful menu actions: add, view, search, update, delete, analyze, save, and load, plus exit. Add, update, and delete save immediately, while the explicit save command and final save on exit provide additional persistence. Search uses a case-insensitive ID comparison. Missing records and invalid actions produce messages instead of exceptions reaching the user.

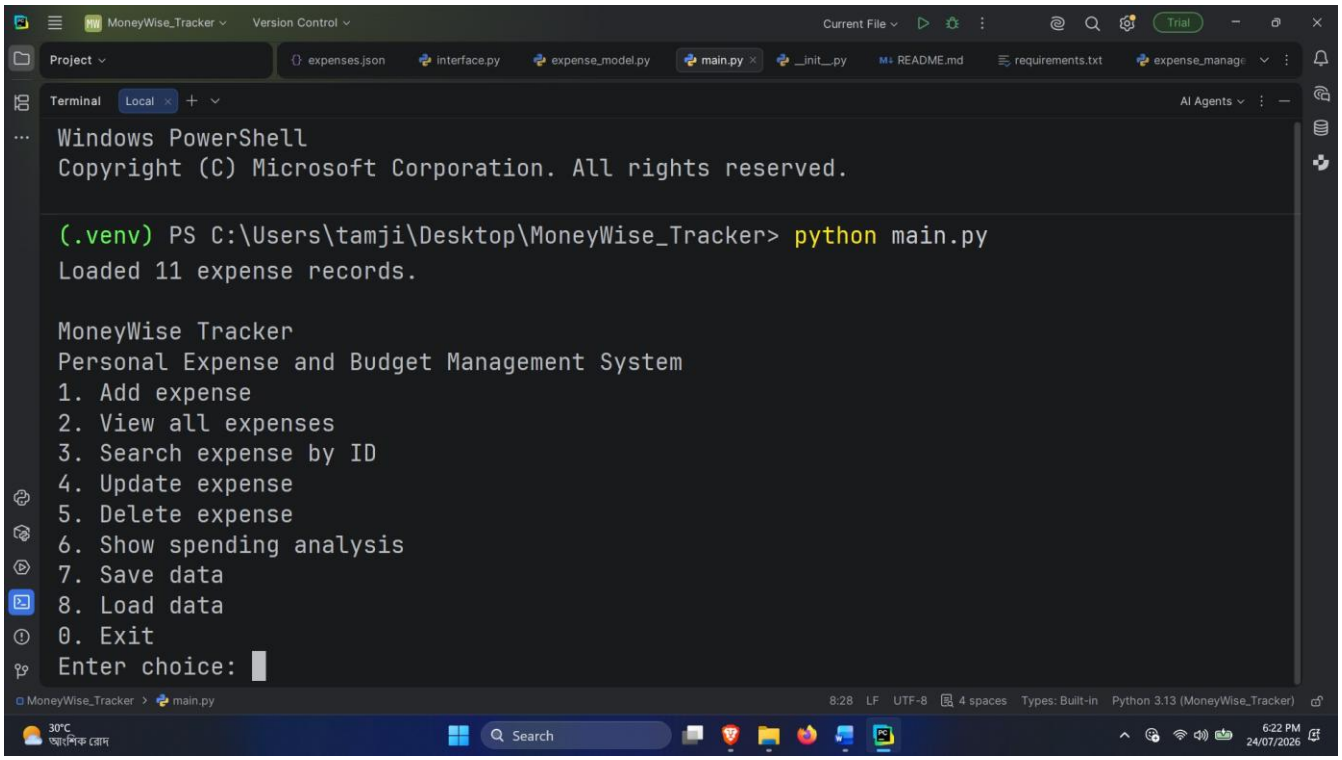

Figure 5. Main menu showing the available user operations.

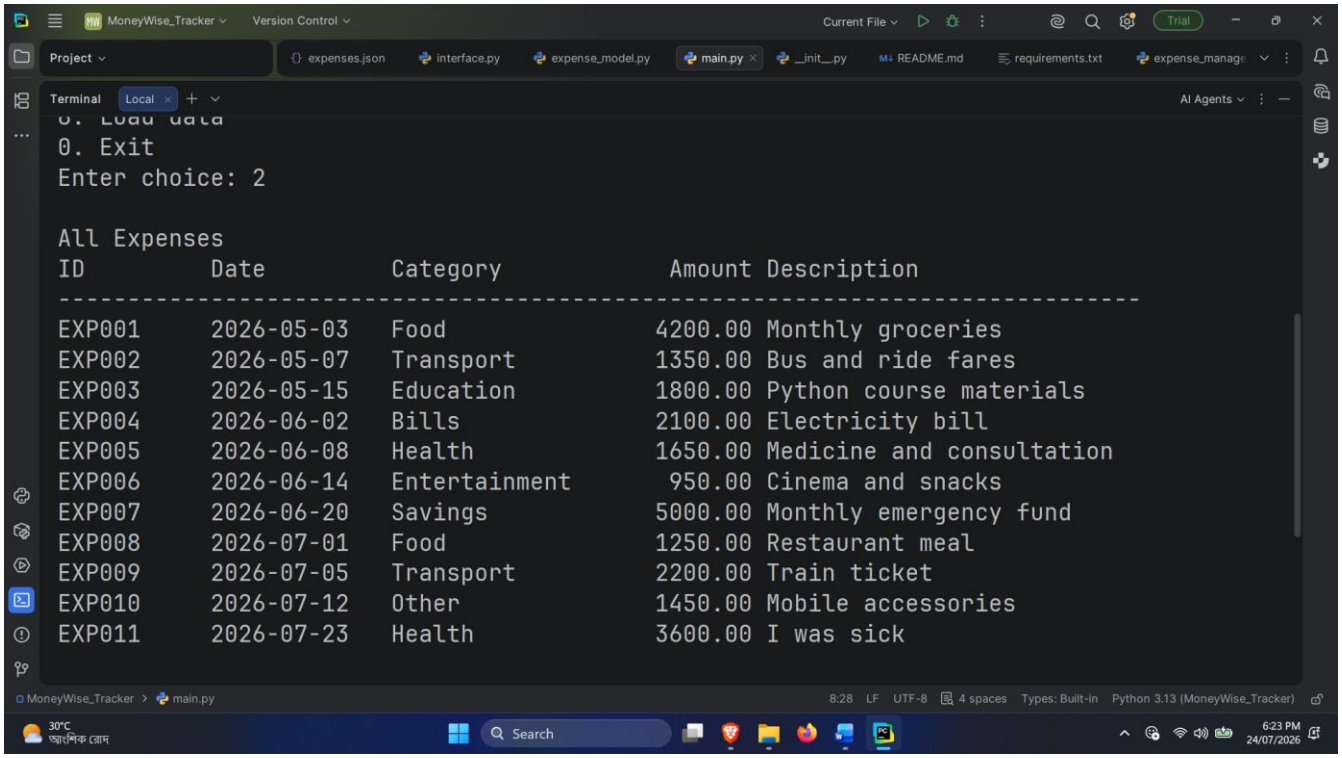

Figure 6. Formatted output for all 10 persistent sample transactions.

The record table demonstrates that values loaded from JSON are reconstructed as Expense objects and displayed in aligned columns. Because the file path is resolved relative to main.py, the same project data file is used even when the program is launched from another working directory.

7.2 Research Question 2: Category and Monthly Spending Patterns

Category totals are built with a dictionary accumulation loop. The results show that Food is the highest category at 5,450.00 (24.83% of total spending), followed by Savings at 5,000.00 (22.78%) and Transport at 3,550.00 (16.17%). Together, these three categories account for 63.78% of all recorded value.

Rank	Category	Total Amount	Share of Overall Total
1	Food	5,450.00	24.83%
2	Savings	5,000.00	22.78%
3	Transport	3,550.00	16.17%
4	Bills	2,100.00	9.57%
5	Education	1,800.00	8.20%
6	Health	1,650.00	7.52%

Rank	Category	Total Amount	Share of Overall Total
7	Other	1,450.00	6.61%
8	Entertainment	950.00	4.33%

Month	Monthly Total	Share of Overall Total	Transactions
2026-05	7,350.00	33.49%	3
2026-06	9,700.00	44.19%	4
2026-07	4,900.00	22.32%	3

June 2026 records the highest monthly total at 9,700.00, representing 44.19% of overall spending. May contributes 7,350.00, while July contributes 4,900.00. The monthly average of the three months that contain records is 7,316.67. This value should not be interpreted as an average over months with no recorded transactions.

7.3 Research Question 3: Descriptive Statistics and Budget Feedback

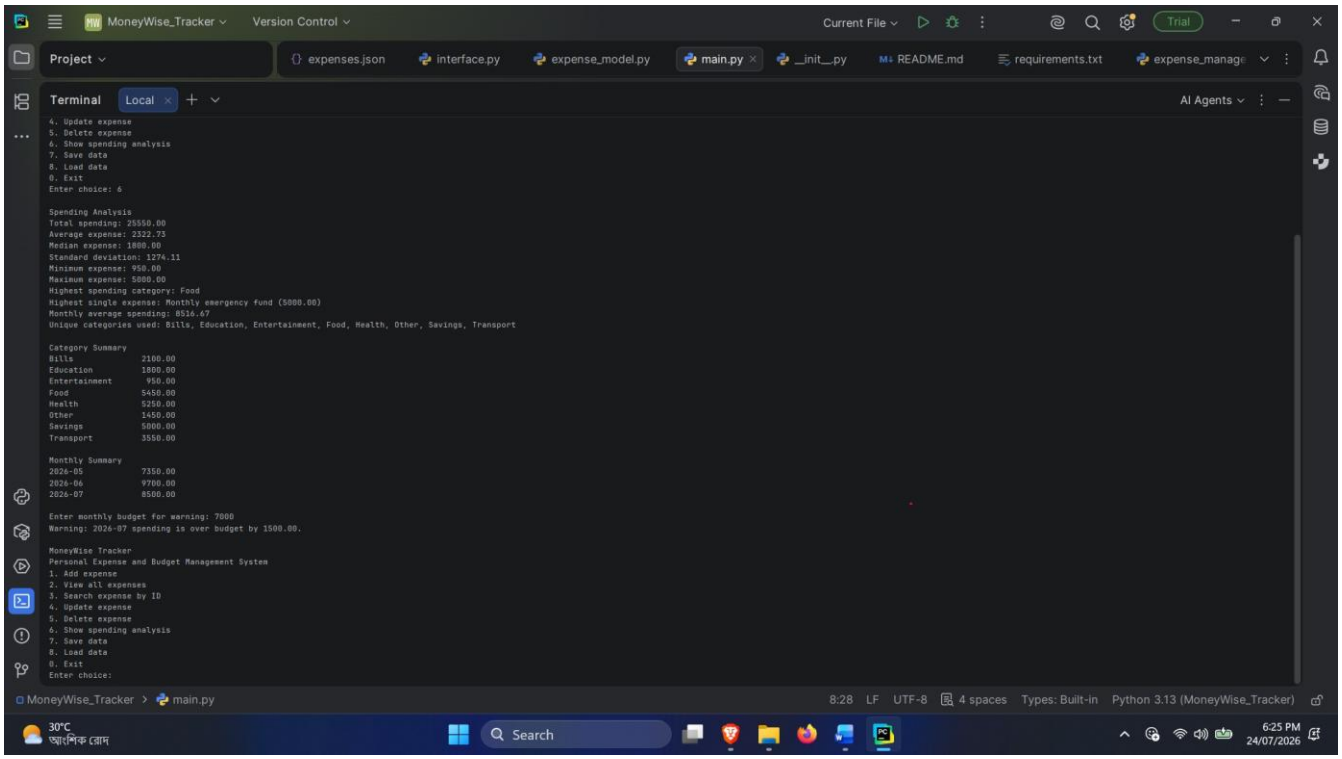

Figure 7. Verified analytical output generated by NumPy and the summary methods.

Metric	Result	Interpretation
Number of expenses	10	Number of valid Expense objects loaded
Total	21,950.00	Sum of all recorded amounts
Mean	2,195.00	Average transaction amount
Median	1,725.00	Middle of the ordered amount values
Population standard deviation	1,267.37	Dispersion calculated with NumPy default ddof=0
Minimum	950.00	Smallest single transaction
Maximum	5,000.00	Largest single transaction
Highest category	Food	Category with greatest accumulated total
Highest single expense	Monthly emergency fund (5,000.00)	Maximum Expense object by amount
Monthly average	7,316.67	Mean of May, June, and July monthly totals

The mean exceeds the median by 470.00 because several larger transactions, especially 4,200.00 and 5,000.00, raise the arithmetic average. The standard deviation of 1,267.37 confirms substantial variation relative to the mean; therefore, a single average alone would not fully describe the spending pattern.

Entered Budget	Latest Recorded Month Total	Utilization	System Message
10,000.00	4,900.00	49.0%	Good: 5,100.00 remains.
6,000.00	4,900.00	81.7%	Careful: spending has reached at least 80%.
4,000.00	4,900.00	122.5%	Warning: spending is over budget by 900.00.

7.4 Functional Requirement Compliance

Requirement	Evidence	Status
FR-1 User interaction	Menu accepts data, action choices, and displays formatted results.	Met
FR-2 Five meaningful operations	Eight operations plus exit are implemented.	Exceeded
FR-3 Persistent data	JSON save/load, autosave after changes, final save on exit.	Met
FR-4 Validation	Empty, malformed, out-of-range, duplicate, and non-finite inputs handled.	Met
FR-5 Analysis or calculation	NumPy statistics, category/month summaries, and budget status.	Exceeded
FR-6 Structured implementation	Separate launcher, interface, model, manager, and data file.	Met
FR-7 Error recovery	Normal mistakes and file failures return messages without crashing.	Met

7.5 Testing and Verification

Verification combined Python compilation, scripted interactive smoke tests, and direct method tests using temporary data files. The following cases were checked against expected behavior.

Test ID	Test Case	Expected / Observed Result	Status
T01	Start with valid sample JSON	Loads 10 records and shows menu	Pass
T02	Start when data file is missing	Creates empty working state; no crash	Pass
T03	Load corrupted JSON	Reports corruption and starts with no records	Pass
T04	Enter blank ID or description	Requests a non-empty value again	Pass
T05	Enter duplicate ID using different letter case	Rejects duplicate	Pass
T06	Enter malformed date	Requests YYYY-MM-DD again	Pass
T07	Enter category outside allowed range	Rejects and repeats category menu	Pass
T08	Enter text, zero, negative, NaN, or infinity as amount	Rejects invalid/non-finite value	Pass
T09	Search existing and missing IDs	Returns record or clear not-found message	Pass
T10	Update an existing record	Validates, changes, and saves it	Pass
T11	Delete an existing record	Removes record and used ID, then saves	Pass
T12	Run NumPy statistics on sample data	Produces verified totals and descriptive results	Pass
T13	Use good, careful, and over-budget thresholds	Returns correct branch message	Pass
T14	Enter an invalid main-menu choice	Shows error and returns to menu	Pass

8. NumPy-Based Custom Computation

NumPy is used for meaningful numerical processing rather than as a decorative import. Expense amounts are converted into a floating-point NumPy array, after which vectorized descriptive functions calculate the central tendency, range, and dispersion of all records. Category and monthly grouping remain explicit Python dictionary logic so that the project continues to demonstrate core data structures.

Component	Description
Computation used	np.sum, np.mean, np.median, np.std, np.min, and np.max on the expense-amount array
Purpose	To summarize total spending, typical transaction size, middle value, variability, and range.
Formula / logic	Mean = $\Sigma x_i/n$; population standard deviation = $\sqrt{[\Sigma(x_i - \text{mean})^2/n]}$. NumPy median uses the middle value(s) after ordering.
Verified result	Total 21,950.00; mean 2,195.00; median 1,725.00; standard deviation 1,267.37; minimum 950.00; maximum 5,000.00.
Main takeaway	The amount distribution is variable and influenced by several larger records, so mean, median, and standard deviation should be interpreted together.

```
amounts = np.array(
    [expense.amount for expense in self.expenses],
    dtype=float,
)

statistics = {
    "total": float(np.sum(amounts)),
    "average": float(np.mean(amounts)),
    "median": float(np.median(amounts)),
    "standard_deviation": float(np.std(amounts)),
    "minimum": float(np.min(amounts)),
    "maximum": float(np.max(amounts)),
}
```

The default np.std calculation uses ddof=0, so the reported value is the population standard deviation of the complete set of records currently held by the application. This implementation choice should be explained during viva because a sample standard deviation would require ddof=1.

9. Key Findings

1. The system successfully manages a complete CRUD workflow and preserves data through JSON, satisfying the core persistence and interaction requirements.
2. All 10 supplied sample records are valid and load successfully; the dataset contains eight unique categories across three months.
3. Overall spending is 21,950.00, with Food contributing the largest category total of 5,450.00 (24.83%).
4. Savings is the second-largest category at 5,000.00 and contains the largest single transaction, “Monthly emergency fund,” at 5,000.00.
5. June 2026 is the highest spending month at 9,700.00, which represents 44.19% of the total value in the sample.
6. The mean transaction amount of 2,195.00 is 470.00 above the median of 1,725.00, indicating that higher transactions pull the average upward.
7. The population standard deviation of 1,267.37 demonstrates that expense amounts are widely dispersed rather than concentrated near one common value.
8. Food, Savings, and Transport together account for 63.78% of total recorded value, so these categories would have the greatest effect on budget planning.
9. The budget function produces three understandable states: good below 80%, careful from 80% through 100%, and warning above the entered budget.
10. Layered validation and exception handling allow the application to recover from expected mistakes and data-file problems without terminating the main loop.

10. Limitations

1. The project is intentionally focused and therefore has several limitations. These limitations do not prevent it from meeting the course requirements, but they define what the present evidence can and cannot support.
2. The interface is command-line based; users do not receive graphical controls or charts.
3. The monthly budget is entered temporarily during analysis and is not stored as a persistent setting.
4. Budget status is calculated for the latest month present in the records, not necessarily the actual current calendar month.
5. The JSON file is appropriate for a small single-user project but does not provide multi-user concurrency, transaction locking, or database-level security.
6. The application tracks expenses only; it does not model income, account balances, recurring transactions, debt, or separate financial accounts.
7. Amounts have no configurable currency symbol, localization, or exchange-rate support.
8. A category tie returns one highest category rather than explicitly reporting every tied category.
9. The 10-record sample is designed for demonstration and cannot establish general behavioral conclusions about personal spending.
10. No authentication or encryption is implemented, so the local JSON file should not be treated as secure storage for sensitive financial information.

11. Possible Future Improvements

- Persist monthly budget values and allow separate budgets for individual categories.
- Add income records, savings goals, recurring expenses, and net balance calculations.
- Provide date-range filtering, sorting, and export to CSV while retaining the present core logic.
- Add a Tkinter interface if approved, without removing the existing validation and manager layers.
- Support tied highest categories and explicit selection of the month to analyze.
- Introduce user authentication or encrypted storage for a production-oriented version.

11. Conclusion

MoneyWise Tracker demonstrates that a relatively small Python application can solve a clear real-life problem while providing strong evidence of fundamental programming competence. The completed system accepts validated expense data, offers more than the minimum number of operations, stores records persistently in JSON, uses meaningful list, tuple, set, and dictionary structures, separates responsibilities across classes and modules, and applies exception handling to both user and file errors. NumPy converts the transaction amounts into informative descriptive statistics, while manual dictionary grouping produces transparent category and monthly summaries. The sample analysis identifies Food as the largest category, June 2026 as the largest month, and a noticeable difference between the mean and median transaction values. Most importantly, the project remains runnable, explainable, and focused enough for demonstration and viva. Within the stated course scope, it is a complete and academically defensible implementation of a personal expense and budget management system.

12. References and Project Sources

Reference	Source
[1]	Programming in Python: Mid-Term Project Requirements, Summer 25-26 Semester. Faculty-provided PDF.
[2]	Final Project Report Template: Project Report Template — Pandas, NumPy, and Matplotlib. Faculty-provided DOCX.
[3]	Python Software Foundation. Python documentation for json, pathlib, datetime, dataclasses, and exception handling.
[4]	NumPy Developers. NumPy reference documentation for array creation and descriptive statistical functions.
[5]	MoneyWise Tracker source package: main.py, expense_model.py, expense_manager.py, interface.py, expenses.json, requirements.txt, and README.md.